

Towards Piecewise Alignment of Vector Embeddings

Ashwin Padaki

December 11, 2025

1 Introduction

The past few years have welcomed the rise of *embeddings*, vector representations that capture the semantics of complex data. Vector embeddings have become a fundamental tool across machine learning and data science [1, 2].

Many embedding models are trained to preserve pairwise relations (dot products or distances). As a result, their spaces are *orthogonally invariant*: any global rotation or reflection leaves those relations unchanged. This property is mathematically natural, but it leads to an issue of *interoperability*: how can one compare or combine embeddings from different models? As an illustrative example, imagine a search system that indexes billions of documents with Model A. If a new, smarter model comes along for embedding user queries (Model B), the two spaces can be separated by an unknown orthogonal transform, so semantically matched query–document pairs no longer align.

This challenge has renewed interest in the classical *Procrustes problem* [3]. In its standard formulation, we are given two sets of vectors

$$\{x_1, \dots, x_n\}, \{y_1, \dots, y_n\} \subset \mathbb{R}^d,$$

and seek an orthogonal transformation $Q \in \mathbb{R}^{d \times d}$ minimizing the squared error

$$\mathcal{L}_2(Q) = \sum_{i=1}^n \|x_i Q - y_i\|_2^2.$$

It is well known that the optimal solution Q^* can be computed efficiently via singular value decomposition. Recent work has revisited this problem in the context of embedding alignment, providing refined error bounds under assumptions such as approximate dot-product preservation [4, 5].

Despite its simplicity, this framework assumes that a *single* rotation is enough to align two embedding spaces. Here, we study a natural generalization of the Procrustes problem where the goal is to find the *piecewise orthogonal alignment* of the vectors. Namely, find a partition of the pairs into k components, along with k orthogonal transformations, that minimizes the alignment error.

In general, these kinds of clustering problems are hard. To obtain a tractable and useful algorithm, we restrict the problem to only consider partitions that are *contiguous along a ordering of the pairs* (e.g., time, 1D projection). This captures settings where the offset between the embedding models is governed by a scalar quantity — it is admittedly a very strong assumption on the data.

Contributions. We give both theoretical and empirical results.

- **Theoretical.** We give a *dynamic programming* algorithm that returns the optimal k -partition (along a given order) and the associated orthogonal maps. The total runtime is $O(n^2 \cdot (d^3 + k))$. The algorithm and analysis are given in Section 2
- **Empirical.** First, we design a synthetic dataset for which the piecewise Procrustes method recovers the segment-wise rotations and vastly reduces alignment loss relative to baseline methods. Runtime results show that the piecewise method is slower but still completes in a reasonable amount of time. Second, we evaluate one downstream task related to retrieval accuracy under a partial model upgrade. The results are mostly negative: we show that the piecewise algorithm only slightly improves upon the original Procrustes algorithm and is prone to overfitting for larger values of k .

2 Theoretical Results

We begin by restating the original Procrustes problem and algorithm [3].

Original (global) Procrustes. Given pairs $\{(x_i, y_i)\}_{i=1}^n$, the *orthogonal Procrustes* problem asks for an orthogonal transformation $Q \in \mathcal{O}(d)$ minimizing

$$\mathcal{L}_2(Q) := \sum_{i=1}^n \|x_i Q - y_i\|_2^2.$$

Equivalently, stacking the vectors in to matrices $X, Y \in \mathbb{R}^{n \times d}$, we have $\mathcal{L}_2(Q) = \|XQ - Y\|_F^2$.

Lemma 2.1 (Section 1 of [3]). *Let $Y^\top X = U\Sigma V^\top$ be a singular value decomposition. Then, $Q^* = VU^\top$ is a minimizer of $\mathcal{L}_2(Q)$.*

Proof. Using $\|M\|_F^2 = \text{Tr}(M^\top M)$, we expand the loss:

$$\mathcal{L}_2(Q) = \|X\|_F^2 + \|Y\|_F^2 - 2 \text{Tr}(Y^\top X Q),$$

so minimizing $\mathcal{L}_2$ is equivalent to maximizing $\text{Tr}(Y^\top X Q)$. Write $Y^\top X = U\Sigma V^\top$ and set $Z := V^\top Q U \in \mathcal{O}(d)$. Then $\text{Tr}(Y^\top X Q) = \text{Tr}(U\Sigma V^\top Q) = \text{Tr}(\Sigma Z)$. Since all $|Z_{ii}| \leq 1$, we have $\text{Tr}(\Sigma Z) \leq \sum_i \sigma_i(\Sigma) = \text{Tr}(\Sigma)$, with equality when $Z = I$. This is achieved by $Q^* = VU^\top$. $\square$

Piecewise Procrustes along a fixed order. Given pairs $\{(x_i, y_i)\}_{i=1}^n$, a fixed permutation $\pi : [n] \rightarrow [n]$, and a parameter $k \in \mathbb{N}$, the (π, k) -*piecewise Procrustes* problem asks for indices $1 = t_1 \leq \dots \leq t_{k+1} = n + 1$ and orthogonal transformations $Q_1, \dots, Q_k \in \mathcal{O}(d)$ minimizing

$$\mathcal{L}_2(\{t_r\}, \{Q^{(r)}\}) = \sum_{r=1}^k \sum_{i=t_r}^{t_{r+1}-1} \|x_{\pi(i)} Q^{(r)} - y_{\pi(i)}\|_2^2$$

where $X^{(r)}, Y^{(r)} \in \mathbb{R}^{m_r \times d}$ are the stacks of the $m_r = t_{r+1} - t_r + 1$ vectors $x_{[t_r, t_{r+1})}$ and $y_{[t_r, t_{r+1})}$, respectively.

Algorithm 1 solves the (π, k) -piecewise Procrustes problem using dynamic programming (DP). We will use the notation $\|M\|_*$ to denote the *nuclear norm* of a matrix M , i.e., the sum of its singular values.

Algorithm 1 (π, k) -Piecewise Procrustes via DP

Require: Paired vectors $\{(x_i, y_i)\}_{i=1}^n \subset \mathbb{R}^d$, ordering permutation π , segments k

- 1: Permute data $(x_i, y_i) \leftarrow (x_{\pi(i)}, y_{\pi(i)})$
 - 2: $P_0 \leftarrow 0_{d \times d}$;
 - 3: **for** $i = 1 \rightarrow n$ **do** $P_i \leftarrow P_{i-1} + y_i x_i^\top$ $\triangleright$ outer product prefix sums
 - 4: **end for**
 - 5: Compute $S[a, b] \leftarrow \|P_b - P_{a-1}\|_*$ for all $1 \leq a \leq b \leq n$ $\triangleright O(n^2 d^3)$
 - 6: **DP Initialization:** $\text{DP}[1, j] \leftarrow S[1, j]$ for all $1 \leq j \leq n$
 - 7: **Backpointers:** $\text{B}[1, j] \leftarrow \{1\}$ for all $1 \leq j \leq n$
 - 8: **for** $\ell = 2 \rightarrow k$ **do**
 - 9: **for** $j = \ell \rightarrow n$ **do**
 - 10: $\text{DP}[\ell, j] \leftarrow \max_{i < j} \{\text{DP}[\ell - 1, i] + S[i, j]\}$ $\triangleright O(n^2 k)$
 - 11: $\text{B}[\ell, j] \leftarrow \text{B}[\ell - 1, i^*] \cup \{i^*\}$ $\triangleright i^*$ is argmax in Line 10
 - 12: **end for**
 - 13: **end for**
 - 14: Compute partition $\{X^{(r)}, Y^{(r)}\}_{r=1}^k$ induced by $\text{B}[k, n]$
 - 15: Let $Q_r = \text{Procrustes}(X^{(r)}, Y^{(r)})$ for each $1 \leq r \leq k$
 - 16: **return** cost $\text{DP}[k, n]$, indices $\text{B}[k, n]$, and transformations $\{Q_r\}_{r=1}^k$
-

Theorem 1 (Correctness of Algorithm 1). *Algorithm 1 finds the optimal solution to the (π, k) -piecewise Procrustes problem, runs in time $O(n^2(d^3 + k))$, and uses $O(n^2 + nd^2)$ space.*

Proof. Fix the permutation π and an index set $1 = t_1 < \dots < t_{k+1} = n + 1$. Write the r -th segment as $I_r = [t_r, t_{r+1})$ and stack $X^{(r)} \in \mathbb{R}^{m_r \times d}, Y^{(r)} \in \mathbb{R}^{m_r \times d}$ from $\{(x_{\pi(i)}, y_{\pi(i)})\}_{i \in I_r}$, where $m_r = t_{r+1} - t_r$. As in the proof of Lemma 2.1,

$$\min_{Q^{(r)} \in O(d)} \|X^{(r)} Q^{(r)} - Y^{(r)}\|_F^2 = \|X^{(r)}\|_F^2 + \|Y^{(r)}\|_F^2 - 2 \|(Y^{(r)})^\top X^{(r)}\|_*.$$

Thus, minimizing the piecewise objective $\mathcal{L}_2(\cdot, \cdot)$ is equivalent to maximizing

$$\sum_{r=1}^k \|(Y^{(r)})^\top X^{(r)}\|_* = \sum_{r=1}^k \left\| \sum_{i \in I_r} y_{\pi(i)} x_{\pi(i)}^\top \right\|_*.$$

Define prefix sums $P_0 := 0$ and $P_j := \sum_{i=1}^j y_{\pi(i)} x_{\pi(i)}^\top$, and scores $S[a, b] := \|P_b - P_{a-1}\|_*$ for $1 \leq a \leq b \leq n$. Then, the optimal k -partition is that which maximizes $\sum_r S[t_r, t_{r+1}]$.

Let $\text{DP}[\ell, j]$ be the optimal score using ℓ segments to cover the prefix $\{1, \dots, j\}$. The last segment must be $[i, j]$ for some $i < j$, so we have the recurrence

$$\text{DP}[\ell, j] = \max_{i < j} \text{DP}[\ell - 1, i] + S[i, j], \quad \text{DP}[1, j] = S[1, j],$$

which exactly gives the base and inductive steps of Algorithm 1. Thus $\text{DP}[k, n]$ equals the optimal score, and the stored backpointers recover the optimal cuts. Given those cuts, setting Q_r to the Procrustes minimizer on segment r (from Lemma 2.1) yields an optimal solution to the original (π, k) -piecewise problem.

It remains to bound the runtime of Algorithm 1. Forming all prefixes P_j takes $O(nd^2)$ time. Computing the $O(n^2)$ interval scores $S[a, b]$ requires one SVD of a $d \times d$ matrix per interval, for a total of $O(n^2 d^3)$ time. The DP layer runs in $O(n^2 k)$ time, and the final k segmentwise Procrustes fits cost $O(kd^3)$. Since $k \leq n$, the overall time is $O(n^2 \cdot (d^3 + k))$. For space, storing all prefixes $\{P_j\}_{j=0}^n$ costs $O(nd^2)$, the interval-score table S costs $O(n^2)$, and the DP/backpointers cost $O(nk)$; hence the total space is $O(n^2 + nd^2)$. $\square$

3 Experimental Evaluation

In this section we empirically evaluate Algorithm 1. We begin with a few important remarks.

- **Ordering requirement.** Piecewise Procrustes assumes a *given one-dimensional ordering* of the pairs and gives the optimal partition on that order. For high-dimensional embeddings, discovering a good scalar order that reflects alignment structure is a difficult, perhaps impossible task. When such a signal is weak or absent, the approach may offer little benefit over a single global transformation. This is a major limitation of our approach.
- **Baseline algorithms.** Aside from using a single global transformation, there is a natural alternative to Piecewise Procrustes that does not rely on an external order: cluster the points (e.g., run k -means on the X vectors) and solve the resulting k instances of Procrustes. We include this algorithm as a competitor: it is more time-efficient (k -means heuristics run much faster than $O(n^2)$ time), but it assumes that alignment is related to proximity in the embedding space.
- **Sampling-based acceleration.** We use datasets of size $n = 10000$, making the $O(n^2)$ runtime of Piecewise Procrustes prohibitive. We restrict cut points to a size- L grid on $[n]$, reducing the time bottleneck to roughly $O(L^2 d^3)$.

3.1 Evaluation on a synthetic dataset

Setup. As a proof of concept, we model a setting where the relationship between two embedding models changes over time in a way that is *locally* well-approximated by a rotation but *globally* piecewise. This is an unrealistically favorable setting for our algorithm, but the goal is to empirically demonstrate guarantee of Theorem 1.

Temporal generator (random walk with jumps). Parameters: Size $N = 10000$, dimension $d = 32$, number of segments $k^* = 4$, small step $\varepsilon = 10^{-5}$, big step $\Gamma = 1$.

1. Draw $X \in \mathbb{R}^{N \times d}$ with rows $x_i \sim \mathcal{N}(0, I_d)$.
2. Split $[1, N]$ into k^* consecutive, equally spaced segments; let the cut indices be $\tau_1 < \dots < \tau_{k^*-1}$.
3. Initialize $Q \in \mathcal{O}(d)$ be a random orthogonal matrix (orthogonal factor of random Gaussian).
4. For $i = 1, \dots, N$:
 - **Small step:** draw $G_i \sim \mathcal{N}(0, 1)^{d \times d}$, set the skew part $S_i := G_i - G_i^\top$, form a near-identity rotation $\delta_i := \text{orth}(I_d + \varepsilon S_i)$, and update

$$Q \leftarrow Q \delta_i, \quad y_i := x_i Q.$$

- **Jump:** if $i \in \{\tau_1, \dots, \tau_{k^*-1}\}$, draw $S \sim \mathcal{N}(0, 1)^{d \times d}$, form $\Delta := \text{orth}(I_d + \Gamma S)$, and update

$$Q \leftarrow Q \Delta.$$

Here $\text{orth}(A)$ denotes the orthogonal component of A (with sign correction to enforce $\det = +1$).

Results. We evaluate three alignment methods: (i) *Global Procrustes*; (ii) *k-means + Procrustes* with $k = 4$ and 300 Lloyd iterations; and (iii) *Piecewise Procrustes* with $k = 4$ and grid size $L = 150$, using the identity (temporal) ordering.

We report runtime (M1 Air, 16GB RAM) and average error.

Method	Time (s)	Average Error
Global	0.0049	97.147
k -means	0.8877	94.496
Piecewise	4.5210	1.183

Table 1: Time and average error for synthetic dataset.

Discussion. Piecewise Procrustes recovers the correct segmentwise rotations and significantly reduces loss relative to both the global and k -means baselines, at the cost of a slightly longer runtime. Global Procrustes fails due to averaging incompatible rotations across segments. Using k -means clusters fail here because the positions of the x_i (random Gaussians) do not correlate with the temporal ordering.

3.2 Evaluation on text embeddings

Setup. Here, we model a downstream task where vector alignment might be useful. We take $N = 10,000$ pairs of Quora questions that are semantically equivalent [6]. The questions in each pair are embedded using two different models: Model A (**all-MiniLM-L6-v2**) and Model B (**paraphrase-MiniLM-L3-v2**).

Questions embedded with Model A are indexed for exact inner-product search [7]. We learn a map from the Model B query space into the Model A index space and evaluate the Recall on held-out pairs. Recall@1 is the most informative (probability the true mate ranks first) but we also unclude Recall@{5, 10}.

Here, we evaluate four alignment methods: *Baseline* (no map), *Global Procrustes*, *k-means + Procrustes* (300 Lloyd iterations), and *Piecewise Procrustes* (grid size $L = 50$). There is no natural ordering to use for the piecewise method, so we use a one-dimensional PCA projection. As a practical upper bound, we report the recall of the true Model A embeddings.

Method	Recall@1 (%)	Recall@5 (%)	Recall@10 (%)
Baseline (no map)	33.60	56.60	67.30
Global Procrustes	59.80	82.40	90.30
k -means + Proc. ($k = 2$)	58.60	82.60	90.00
k -means + Proc. ($k = 10$)	56.40	82.20	90.10
PCA Piecewise ($k = 2$)	59.00	83.00	90.30
PCA Piecewise ($k = 10$)	55.50	79.40	89.10
Target (Model A)	62.80	83.90	90.40

Table 2: Retrieval accuracy for baselines, k -means and PCA piecewise ($k = \{2, 10\}$)

Method	Train MSE	Test MSE
Baseline (no map)	1.470	1.470
Global Procrustes	0.637	0.657
k -means ($k = 2$)	0.575	0.622
k -means ($k = 10$)	0.391	0.562
PCA Piecewise ($k = 2$)	0.576	0.622
PCA Piecewise ($k = 10$)	0.432	0.665

Table 3: Residual (MSE) on train/test. Lower is better.

Discussion. All alignment methods substantially improve over the baseline and approach the target model’s recall. Indeed, the Piecewise method matches (with $k = 2$) matches Global at R@10 and is slightly only higher at R@5. Interestingly, increasing the partition size to $k = 10$ degrades performance and widens the train-test residual gap, which perhaps indicates overfitting to the noisy 1-D PCA. Notably, k -means attains the lowest *test* MSE among the learned maps, yet its recall trails that of Global Procrustes.

References

- [1] T. Mikolov, K. Chen, G. Corrado, and J. Dean, *Efficient estimation of word representations in vector space* (2013), **1301.3781**, URL <https://arxiv.org/abs/1301.3781>.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, *Bert: Pre-training of deep bidirectional transformers for language understanding* (2019), **1810.04805**, URL <https://arxiv.org/abs/1810.04805>.
- [3] P. H. Schönemann, *Psychometrika* **31**, 1 (1966), ISSN 1860-0980, URL <https://doi.org/10.1007/BF02289451>.
- [4] L. Maystre, A. O. Gonzalez, C. Park, R. Dolga, T. Berariu, Y. Zhao, and K. Ciosek, *When embedding models meet: Procrustes bounds and applications* (2025), **2510.13406**, URL <https://arxiv.org/abs/2510.13406>.
- [5] K. Zielnicki and K.-J. Hsiao, in *Proceedings of the Nineteenth ACM Conference on Recommender Systems* (ACM, 2025), RecSys '25, p. 1030–1033, URL <http://dx.doi.org/10.1145/3705328.3748141>.
- [6] DataCanary, hilfalkaff, L. Jiang, M. Risdal, N. Dandekar, and tomtung, *Quora question pairs*, <https://kaggle.com/competitions/quora-question-pairs> (2017), kaggle.
- [7] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, *The faiss library* (2025), **2401.08281**, URL <https://arxiv.org/abs/2401.08281>.